



UNIVERSITÀ DEGLI STUDI DI MESSINA

DIPARTIMENTO DI SCIENZE MATEMATICHE E INFORMATICHE, SCIENZE
FISICHE E SCIENZE DELLA TERRA

Corso di Laurea Triennale in Informatica

Identificazione e verifica UBO
BaseX vs Neo4j

Docente:
Prof. Massimo Villari
Prof. Antonio Celesti

Realizzato da:
Davide Ferrara

Introduzione

Obiettivo del Progetto

Questo progetto si propone di analizzare e confrontare le funzionalità di due sistemi di gestione dei database, Neo4j e BaseX, focalizzandosi sulla loro efficacia nell'elaborazione di informazioni relative al Beneficiario Effettivo Ultimo (UBO). Pur non essendo finalizzato alla reale identificazione degli UBO secondo quanto previsto dai criteri del progetto, lo studio si basa sulla creazione di un dataset che rappresenta scenari realistici di strutture societarie complesse. A partire da questo dataset simulato, il progetto intende eseguire cinque interrogazioni con livelli crescenti di complessità su entrambi i database, al fine di confrontarne le prestazioni in termini di velocità di esecuzione e capacità di gestione di dati articolati.

Tecnologie Utilizzate

Per affrontare la sfida della gestione e interrogazione dei dati legati ai Beneficiari Effettivi Ultimi (UBO), sono stati selezionati due database che incarnano approcci differenti nella modellazione e nel trattamento delle informazioni:

Neo4j: Si tratta di un database a grafo che organizza i dati sotto forma di nodi, relazioni e attributi. Questo modello si presta particolarmente bene alla rappresentazione di reti complesse e intrecciate, come quelle riscontrabili nelle strutture di proprietà aziendale multilivello tipiche dell'analisi UBO. Grazie alla sua architettura orientata ai grafi, Neo4j consente una navigazione efficiente tra entità interconnesse.

BaseX: È un database nativo per XML e utilizza XQuery come linguaggio di interrogazione. È progettato per gestire dati altamente strutturati e gerarchici, qualità che lo rendono adatto alla modellazione delle catene di proprietà in formato XML. In questo progetto, BaseX offre la possibilità di rappresentare strutture societarie complesse e di esplorarle attraverso interrogazioni avanzate basate su XQuery.

Per automatizzare le operazioni di caricamento, interrogazione e analisi dei dati su entrambe le piattaforme, è stato impiegato il linguaggio **Python**. Grazie a librerie dedicate come **py2neo** per Neo4j e **BaseXClient.py** per BaseX, è stato possibile gestire in modo agevole la connessione ai database, l'esecuzione di query e la raccolta dei risultati per confrontarne l'efficacia.

Importanza dello Studio

L'identificazione e la verifica del Beneficiario Effettivo Ultimo (UBO) sono diventate attività essenziali per garantire la conformità normativa nelle istituzioni finanziarie e in numerosi altri settori regolamentati a livello globale. Garantire la trasparenza nella titolarità delle imprese è infatti un elemento chiave nella lotta contro il riciclaggio di denaro, la corruzione e altri reati finanziari. Capire chi esercita il controllo effettivo su una persona giuridica permette alle organizzazioni di valutare e ridurre i rischi associati a transazioni finanziarie e rapporti d'affari.

Negli ultimi anni, normative internazionali come il Corporate Transparency Act negli Stati Uniti e le direttive europee antiriciclaggio (4AMLD e 5AMLD) hanno posto un'enfasi crescente sull'importanza di mantenere informazioni aggiornate, chiare e accessibili sui beneficiari effettivi. Tuttavia, l'applicazione concreta di queste norme richiede strumenti tecnologici in grado di affrontare la complessità delle strutture societarie articolate e multilivello.

All'interno di questo scenario, il presente studio si propone di confrontare due sistemi di gestione dei dati molto differenti tra loro: Neo4j, un database a grafo, e BaseX, un database nativo XML. L'obiettivo è valutare quale dei due sia più adatto a trattare strutture dati complesse, analoghe a quelle coinvolte nell'identificazione degli UBO. Neo4j risulta particolarmente promettente per questo compito grazie al suo approccio orientato ai grafi, ideale per rappresentare e interrogare reti di relazioni. D'altra parte, BaseX, grazie alla sua gestione efficiente di dati XML e alla potenza espressiva di XQuery, rappresenta una valida alternativa, in grado di offrire flessibilità e scalabilità.

Descrizione del Processo

Il progetto prende avvio con la costruzione di un dataset simulato, pensato per riprodurre fedelmente strutture di proprietà aziendale complesse, tipiche dei contesti in cui è necessario identificare il Beneficiario Effettivo Ultimo (UBO). Il dataset è costituito da nodi che rappresentano entità giuridiche e da relazioni che modellano i legami di proprietà e controllo tra queste entità.

Una volta definito, il dataset viene caricato nei due database oggetto di studio, Neo4j e BaseX, attraverso script Python automatizzati. L'utilizzo di Python consente un'integrazione efficace e flessibile, semplificando le operazioni di inserimento dati, configurazione delle strutture e interazione con le rispettive API (py2neo per Neo4j, BaseXClient.py per BaseX).

Successivamente, vengono sviluppate cinque query con complessità crescente, progettate per mettere alla prova diverse capacità dei database: dalla gestione di relazioni su più livelli, all'esecuzione di interrogazioni complesse su grandi quantità di dati. Ogni query mira a valutare le prestazioni sotto aspetti specifici, come la profondità della navigazione tra entità, la velocità di risposta e la gestione efficiente della struttura informativa.

I dati ottenuti dall'esecuzione delle query su entrambi i sistemi vengono poi raccolti e analizzati in modo comparativo. L'analisi finale fornisce una valutazione dettagliata delle performance di Neo4j e BaseX, evidenziandone vantaggi e limitazioni. In conclusione, il progetto propone indicazioni pratiche su quale delle due soluzioni risulti più adatta per applicazioni che richiedono la gestione di strutture dati complesse, come quelle implicate nell'identificazione degli UBO.

Descrizione dei Database

Introduzione ai Database Utilizzati

Neo4j

Neo4j è uno dei più diffusi database a grafo disponibili sul mercato, progettato specificamente per gestire e interrogare dati con relazioni complesse. Questa caratteristica lo rende particolarmente adatto per casi d'uso che coinvolgono connessioni intricate tra entità, come quelle tipiche delle strutture di proprietà aziendale necessarie per l'identificazione degli UBO.

Architettura e Modello dei Dati

Neo4j adotta un modello di dati basato sui grafi, dove le informazioni sono rappresentate tramite nodi, relazioni e proprietà:

- **Nodi:** Rappresentano entità o oggetti nel dominio dell'applicazione, come persone, aziende, shareholders o transazioni.
- **Relazioni:** Definiscono le connessioni tra i nodi. Ogni relazione ha una direzione, peso o un tipo che definisce il significato della connessione, come "owns" o "share".
- **Proprietà:** Sia i nodi che le relazioni possono avere attributi chiave-valore che forniscono informazioni aggiuntive, come nomi, date o valori numerici.

Cypher:

Neo4j utilizza Cypher come linguaggio di query, un linguaggio dichiarativo ispirato all'ascii-art che eredita parte della sintassi da SQL per mantenere una continuità rispetto i database relazionali.

Le sue caratteristiche principali sono:

- **Pattern Matching:** Permette di descrivere i percorsi nei grafi usando una sintassi visiva (ascii-art) che rende facile capire le query a colpo d'occhio.
- **Aggregazione e Funzioni di Analisi:** Supporta operazioni di aggregazione sui nodi e le relazioni, oltre a funzioni di analisi avanzate.
- **Facilità d'Uso:** La sintassi di Cypher è intuitiva e consente agli utenti di esprimere query complesse con meno codice rispetto ad altri linguaggi di programmazione.

BaseX

Architettura e Modello dei Dati

In BaseX, i dati vengono organizzati in documenti XML, che rappresentano la struttura gerarchica delle informazioni. Ogni elemento XML può contenere sotto-elementi, attributi e valori di testo, permettendo una modellazione precisa di scenari complessi come le catene di proprietà multilivello tipiche dell'identificazione degli UBO. I dati vengono rappresentati in:

- **Documenti XML:** Ogni documento rappresenta una struttura di dati gerarchica, simile a un albero, con elementi, attributi e valori di testo.
- **Collezioni di Documenti:** I documenti XML possono essere raggruppati in collezioni, il che permette di organizzare e interrogare insiemi di dati correlati in modo efficiente.

XQuery:

BaseX utilizza **XQuery**, uno standard per interrogare e manipolare documenti XML.

Le caratteristiche principali di XQuery sono:

- **Query su Dati Gerarchici:** Permette di navigare e interrogare i dati XML come un albero, accedendo agli elementi e ai loro sottoelementi con una sintassi specifica.
- **Filtraggio e Trasformazione dei Dati:** Supporta operazioni di filtraggio e trasformazione dei dati XML, consentendo l'estrazione e la modifica di parti specifiche dei documenti.
- **Aggregazione e Funzioni Avanzate:** XQuery offre una vasta gamma di funzioni per l'aggregazione e l'elaborazione dei dati, simile a quanto offerto nei tradizionali linguaggi di query.

Struttura del Database

Schema dei Dati

Collezioni e Nodi

1. Aziende (Companies):

- **Descrizione:** Rappresentano le aziende e forniscono informazioni dettagliate su ciascuna entità aziendale.
- **Campi:** Name, address, legal_form, registration_details e financial_data.

- **Riferimenti:** IDs of administrators, shareholders, ubo, transactions.
- 2. **Amministratori (Administrators):**
 - **Descrizione:** Chi amministra le aziende.
 - **Campi:** Name, address, birthdate, nationality.
- 3. **Azionisti (Shareholders):**
 - **Descrizione:** Persone o le entità che possiedono azioni nelle aziende.
 - **Campi:** Name, type, ownership_percentage, address, birthdate e nationality
- 4. **Beneficiari Effettivi (UBO - Ultimate Beneficial Owners):**
 - **Descrizione:** Rappresentano i proprietari ultimi delle aziende, con un focus specifico su chi beneficia effettivamente delle proprietà.
 - **Campi:** Name, address, birthdate, nationality, ownership_percentage e type.
- 5. **Transazioni (Transactions):**
 - **Descrizione:** Rappresentano le transazioni finanziarie effettuate dalle aziende.
 - **Campi:** Type, amount, date, currency.
- 6. **Controlli KYC/AML (KYC_AML_Checks):**
 - **Descrizione:** Rappresentano i controlli di conformità per la verifica dei beneficiari effettivi tramite il “Know Your Customer / Anti-Money Laundering Checks”
 - **Campi:** Type, result, date e notes.
 - **Riferimenti:** ubo_id.

BaseX

Ecco come sono state strutturate le relazioni in BaseX:

1. Relazioni Implicite:

Ho considerato l'entità **companies** come entità principale nel contesto del nostro database BaseX. Possiamo descrivere le relazioni tra l'entità **companies** e le altre entità del database attraverso riferimenti ID. In questo modello, le relazioni sono gestite attraverso elementi XML all'interno dei documenti di **companies** che contengono ID di documenti di altre collezioni.

Tutte le relazioni nel database sono di tipo **uno-a-molti**. Questo modello di relazioni è stato scelto per la flessibilità e la scalabilità nella gestione dei dati. In questo contesto, un'azienda può essere associata a molte altre entità subordinate (come amministratori, azionisti, transazioni, ecc.).

Relazione con Administrators:

- **Descrizione:** Ogni azienda ha uno o più amministratori che la gestiscono.
- **Dettagli della Relazione:** Un documento **company** può includere un array di ID che fanno riferimento a documenti **administrators**. Questo tipo di relazione consente di gestire facilmente l'elenco di amministratori di ogni azienda e di mantenere aggiornate le informazioni relative.

Relazione con Shareholders:

- **Descrizione:** Le aziende hanno azionisti che possiedono quote della società.
- **Dettagli della Relazione:** L'azienda può avere un array di ID che puntano a documenti **shareholders**. Questo approccio consente di rappresentare con precisione le proprietà azionarie delle aziende.

Relazione con UBO (Ultimate Beneficial Owners):

- **Descrizione:** I beneficiari effettivi sono coloro che, direttamente o indirettamente, traggono vantaggio dalla proprietà dell'azienda.
- **Dettagli della Relazione:** L'azienda tiene traccia degli ID dei beneficiari effettivi nei suoi documenti. La struttura consente di mappare chiaramente chi beneficia dalla proprietà di un'azienda.

Relazione con Transactions:

- **Descrizione:** Le aziende effettuano transazioni finanziarie che devono essere registrate.
- **Dettagli della Relazione:** L'azienda può avere numerose transazioni finanziarie. Questo permette di tracciare tutte le attività finanziarie di un'azienda in modo dettagliato.

Relazione con KYC AML Checks:

- **Descrizione:** Le aziende devono effettuare controlli di conformità KYC/AML per prevenire il riciclaggio di denaro e il finanziamento del terrorismo.
- **Dettagli della Relazione:** L'azienda può avere molti controlli KYC/AML associati. Questo modello di relazione supporta il monitoraggio e la gestione efficiente delle attività di conformità.

Neo4j

2. Relazioni:

- **COMPANY_HAS_ADMINISTRATOR):**
Relazione tra un'azienda e i suoi amministratori, includendo il ruolo e le date di inizio e fine.
- **COMPANY_HAS_SHAREHOLDER:** Relazione tra un'azienda e i suoi azionisti, con proprietà come la percentuale di partecipazione e la data di acquisto.
- **COMPANY_HAS_UBO:** Relazione tra un'azienda e i suoi beneficiari effettivi, simile a quella con gli azionisti.
- **COMPANY_HAS_TRANSACTION:** Relazione tra un'azienda e le sue transazioni.
- **UBO_HAS_CHECKS:** Relazione tra un beneficiario effettivo e i controlli KYC/AML associati.

Processi di Popolamento dei Database

I dati per entrambi i database sono stati generati utilizzando la libreria Python “**Faker**”, che ha permesso di creare dati realistici ma fittizi, come nomi, indirizzi e date.

Generazione dei Dati

- **Administrators:** Sono stati generati 2500 record, inclusivi di dettagli come nome, indirizzo, data di nascita e nazionalità.
- **Shareholders:** Sono stati creati 2500 record con informazioni simili, ma includendo anche il tipo di azionista e la percentuale di proprietà.
- **Ubo (Ultimate Beneficial Owners):** 5.000 record sono stati generati, comprendendo dettagli simili agli azionisti.
- **Transactions:** 200.000 record di transazioni sono stati creati con dettagli come tipo, importo, data e valuta.
- **KYC_AML_Checks:** 15.000 record sono stati generati per i controlli di verifica e conformità.
- **Companies:** 25.000 record sono stati creati con dettagli come nome, indirizzo, forma legale, dati finanziari e riferimenti ad amministratori, azionisti, UBO, transazioni e controlli KYC/AML.

I dati generati sono stati esportati in file CSV, e importati nel database utilizzando uno script Python

Dati:

- ``administrators.csv``
- ``shareholders.csv``
- ``ubo.csv``
- ``transactions.csv``
- ``kyc_aml_checks.csv``
- ``companies.csv``

Implementazione

Questa sezione descrive le query sviluppate per interagire con il database e analizzare le informazioni contenute al suo interno. Le interrogazioni sono state concepite per raccogliere, trasformare e interpretare dati riguardanti società, relazioni con amministratori, shareholders, beneficiari finali, movimenti finanziari e controlli di conformità KYC/AML.

Ogni query è pensata per simulare output utili all'esplorazione delle strutture di proprietà e delle operazioni aziendali.

QUERY 1

Descrizione: Questa query è progettata per estrarre i dettagli di un'azienda specifica utilizzando l'identificativo unico (ID) dell'azienda.

Querye tempi di esecuzione:

- [BaseX Query 1](#)
- [NEO4J QUERY 1](#)

QUERY 2

Descrizione: Questa query recupera informazioni dettagliate su un'azienda specifica e sui suoi amministratori associati. Inizia cercando l'azienda in base al suo ID e, successivamente, raccoglie i dettagli degli amministratori associati all'azienda.

Querye tempi di esecuzione:

- [BaseX Query 2](#)
- [NEO4J QUERY 2](#)

QUERY 3

Descrizione: Questa query estende la Query 2 (per aumentare il costo) includendo anche i dettagli degli UBO (Ultimate Beneficial Owners) che possiedono più del 25% dell'azienda.

Querye tempi di esecuzione:

- [BaseX Query 3](#)
- [NEO4J QUERY 3](#)

QUERY 4

Descrizione: Questa query fornisce una vista dettagliata dell'azienda, inclusi amministratori, UBO con più del 25% di proprietà e in fine la somma degli importi delle transazioni in un intervallo di date specifico.

Querye tempi di esecuzione:

- [BaseX Query 4](#)
- [NEO4J QUERY 4](#)

QUERY 5

Descrizione: Questa Query estende la Query 4 e ritorna insieme agli amministratori, UBO con più del 25% di proprietà e l'amount totale di tutte le transazioni in un certo intervallo di date, tutti gli shareholders e le distinte transazioni.

Querye tempi di esecuzione:

- [BaseX Query 5](#)
- [NEO4J QUERY 5](#)

Conclusione

Il benchmark ha confrontato le prestazioni di **Neo4j** e **BaseX** nell'esecuzione di 5 query di complessità crescente, la quale aumentava aggiungendo progressivamente una " *join*" per ciascuna query. Le query sono state eseguite su 4 dataset di dimensioni diverse, rispettivamente: 100%, 75%, 50% e 25%, dove il dataset al 100% conteneva 250.000 (sia su Neo4j sia su BaseX).

I risultati hanno evidenziato che Neo4j ha sempre superato BaseX in termini di velocità di esecuzione, indipendentemente dalla dimensione del dataset.

La differenza di prestazioni è risultata ancora più marcata all'aumentare della complessità delle query, sottolineando la maggiore efficienza di Neo4j nella gestione di operazioni di " *join*" complesse, grazie alla sua architettura orientata ai grafi. Pertanto, Neo4j si dimostra più adatto a gestire query con molti collegamenti tra i dati, anche su dataset di dimensioni variabili.